



Concurrent Programming & Multiprocessing/Multithreading

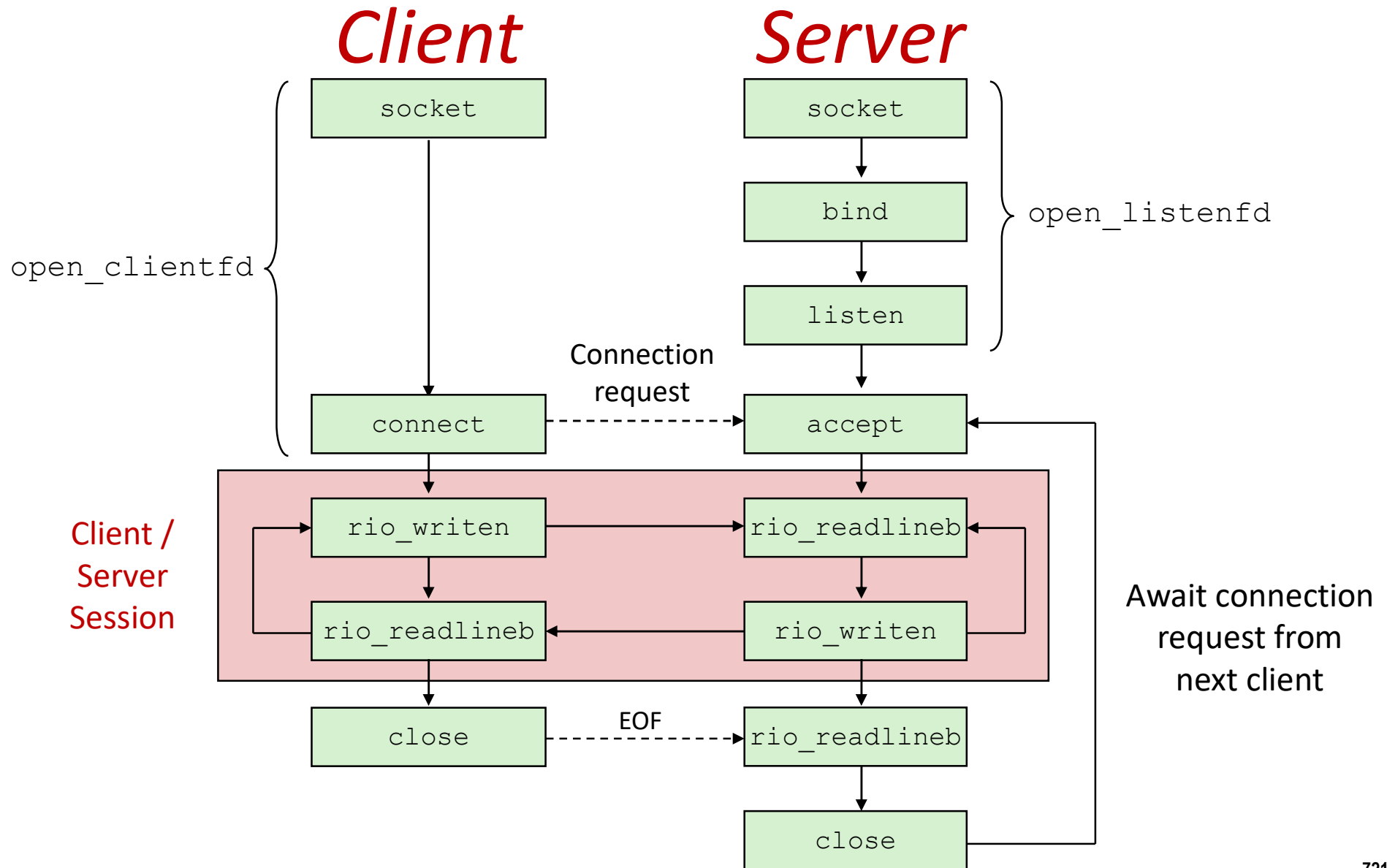
Concurrent Programming is Hard!

- The human mind tends to be sequential
- The notion of time is often misleading
- Thinking about all possible sequences of events in a computer system is at least error prone and frequently impossible

Concurrent Programming is Hard!

- **Classical problem classes of concurrent programs:**
 - ***Races***: outcome depends on arbitrary scheduling decisions elsewhere in the system
 - Example: who gets the last seat on the airplane?
 - ***Deadlock***: improper resource allocation prevents forward progress
 - Example: traffic gridlock
 - ***Livelock / Starvation / Fairness***: external events and/or system scheduling decisions can prevent sub-task progress
 - Example: people always jump in front of you in line
- **Many aspects of concurrent programming are beyond the scope**

Iterative Echo Server



Server concurrency (3 approaches)

Allow server to handle multiple clients simultaneously

■ 1. Processes

- Kernel automatically interleaves multiple logical flows
- Each flow has its own private address space

■ 2. Threads

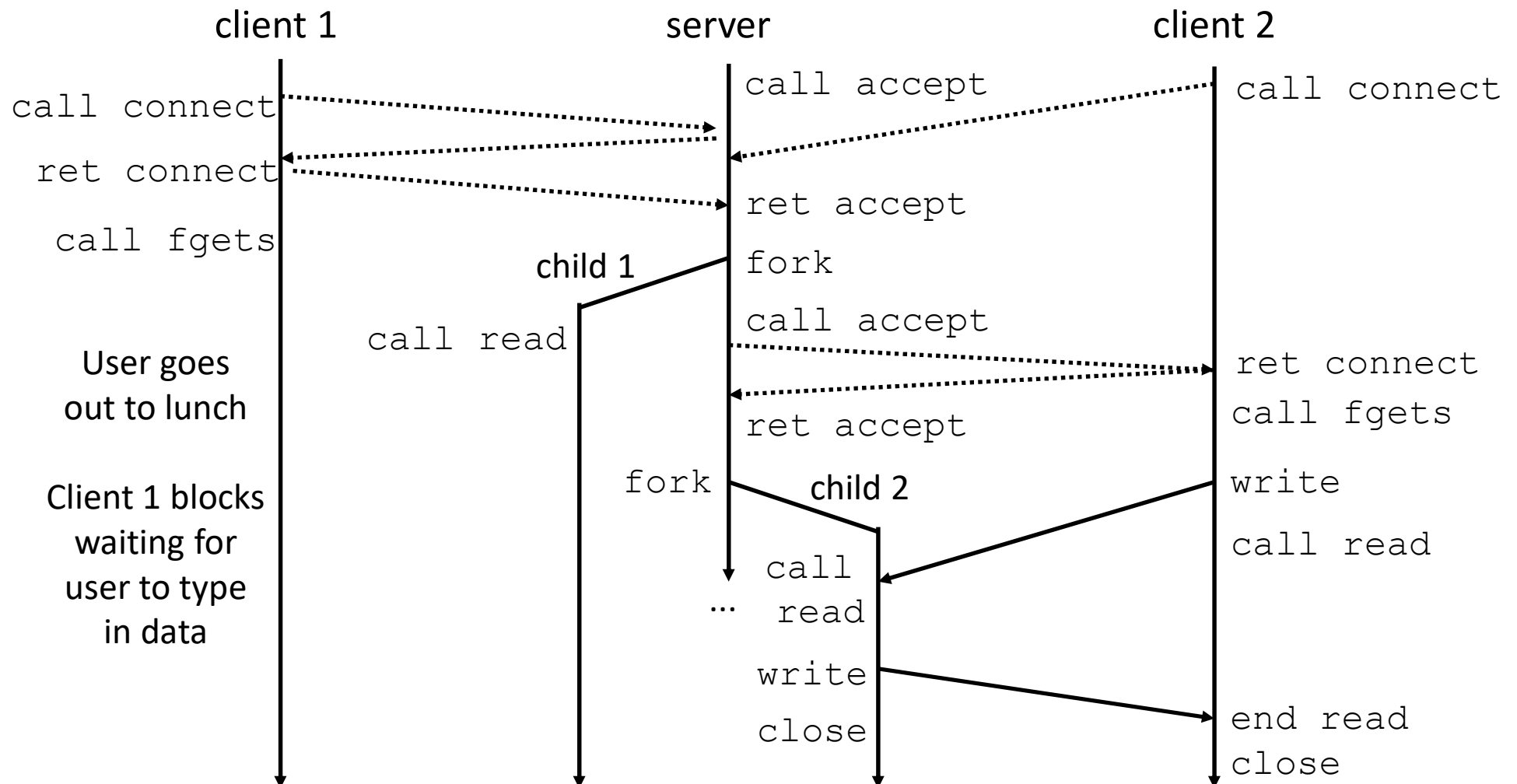
- Kernel automatically interleaves multiple logical flows
- Each flow shares the same address space

■ 3. I/O multiplexing with `select()`

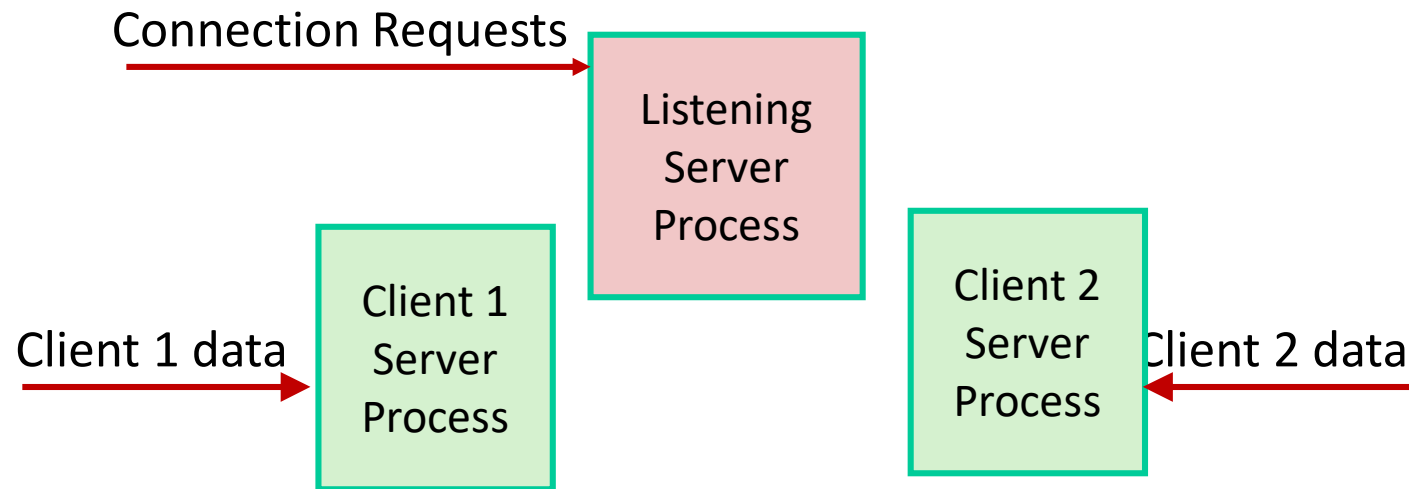
- Programmer manually interleaves multiple logical flows
- All flows share the same address space
- Relies on lower-level system abstractions

Concurrent Servers: Multiple Processes

- Spawn separate process for each client



Process Execution Model

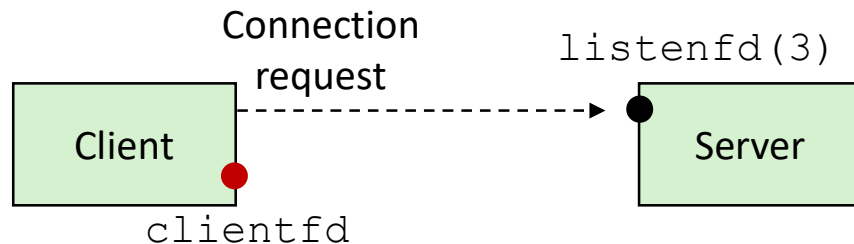


- Each client handled by independent process
- No shared state between them
- Both parent & child have copies of listenfd and connfd
 - Parent must close connfd
 - Child must close listenfd

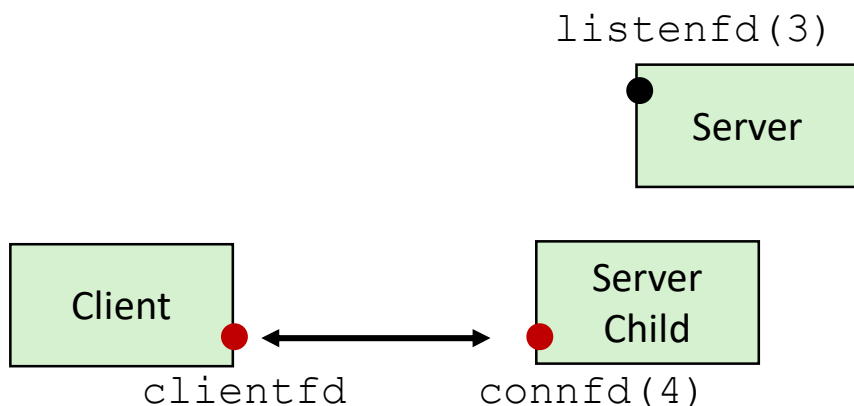
Concurrent Server: `accept` Illustrated



1. Server blocks in `accept`,
waiting for connection
request on listening descriptor
`listenfd`



2. Client makes connection
request by calling `connect`



3. Server returns `connfd` from
`accept`. Forks child to handle client.
Connection is now established
between `clientfd` and `connfd`

Implementation Must-dos With Process-Based Designs

- **Listening server process must reap zombie children**
 - to avoid fatal memory leak
- **Listening server process must close its copy of `connfd`**
 - Kernel keeps reference for each socket/open file
 - After fork, `refcnt(connfd) = 2`
 - Connection will not be closed until `refcnt(connfd) == 0`

Pros and Cons of Process-Based Designs

- **+ Handle multiple connections concurrently**
- **+ Clean sharing model**
 - descriptors (no)
 - file tables (yes)
 - global variables (no)
- **+ Simple and straightforward**
- **– Additional overhead for process control**
- **– Nontrivial to share data between processes**
 - Requires IPC (interprocess communication) mechanisms
 - FIFO's (named pipes), System V shared memory and semaphores

Approach #2: Multiple Threads

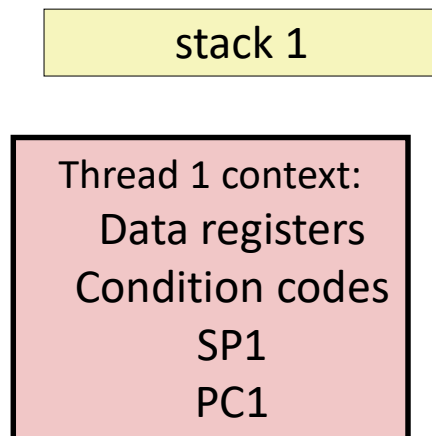
- **Very similar to approach #1 (multiple processes)**
 - but, with threads instead of processes

A Process With Multiple Threads

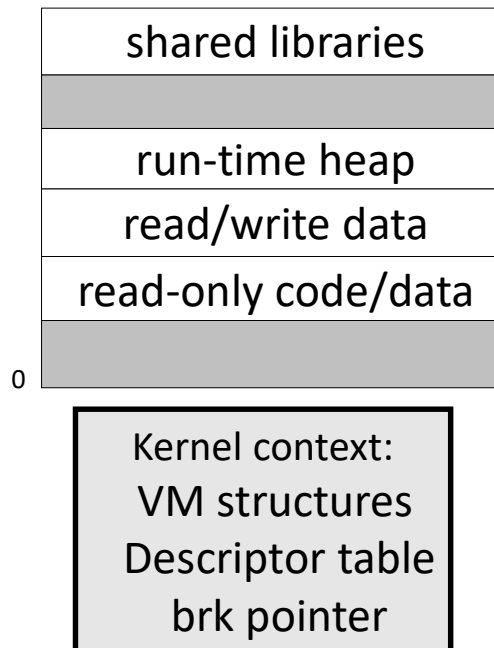
■ Multiple threads can be associated with a process

- Each thread has its own logical control flow
- Each thread shares the same code, data, and kernel context
 - Share common virtual address space (inc. stacks)
- Each thread has its own thread id (TID)

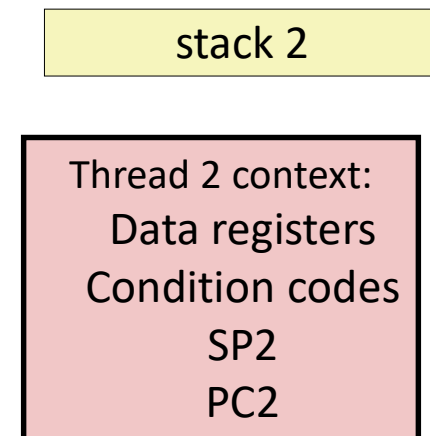
Thread 1 (main thread)



Shared code and data



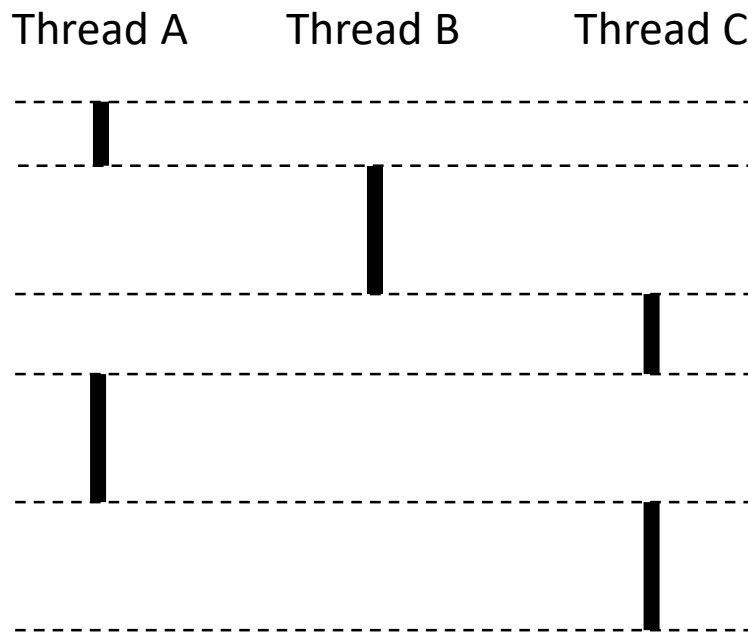
Thread 2 (peer thread)



Thread Execution

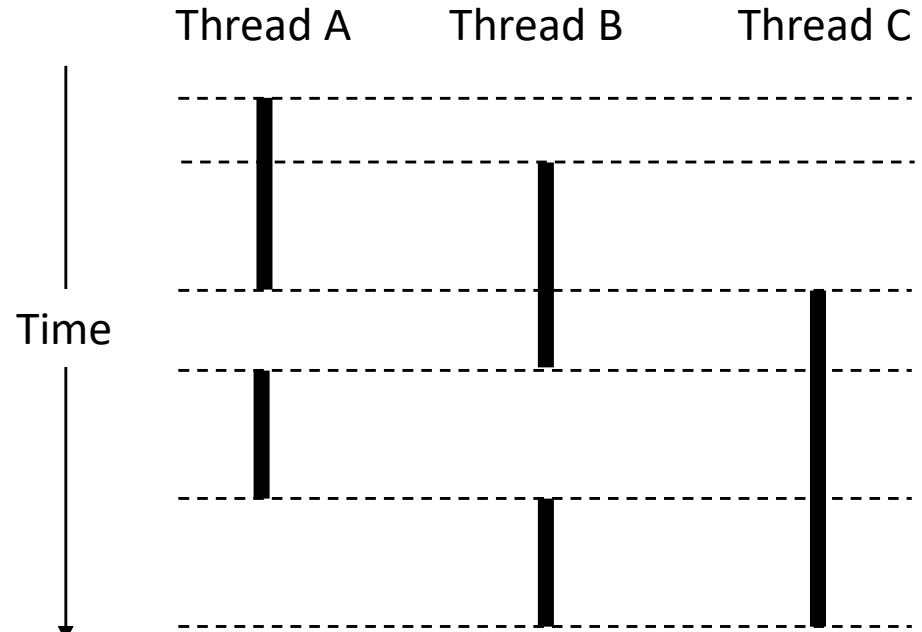
■ Single Core Processor

- Simulate concurrency by time slicing



■ Multi-Core Processor

- Can have true concurrency



Run 3 threads on 2 cores

Threads vs. Processes

■ How threads and processes are similar

- Each has its own logical control flow
- Each can run concurrently with others (possibly on different cores)
- Each is context switched

■ How threads and processes are different

- Threads share code and some data
 - Processes (typically) do not
- Threads are somewhat less expensive than processes
 - Process control (creating and reaping) twice as expensive as thread control
 - Linux numbers:
 - ~20K cycles to create and reap a process
 - ~10K cycles (or less) to create and reap a thread

Issues With Thread-Based Servers

- **Must run “detached” to avoid memory leak**
 - At any point in time, a thread is either *joinable* or *detached*
 - *Joinable* thread can be reaped and killed by other threads
 - must be reaped (with `pthread_join`) to free memory resources
 - *Detached* thread cannot be reaped or killed by other threads
 - resources are automatically reaped on termination
 - Default state is joinable
 - use `pthread_detach(pthread_self())` to make detached
- **Must be careful to avoid unintended sharing**
 - For example, passing pointer to main thread's stack
 - `Pthread_create(&tid, NULL, thread, (void *)&connfd);`
- **All functions called by a thread must be *thread-safe***
 - (next lecture)

Pros and Cons of Thread-Based Designs

- **+ Easy to share data structures between threads**
 - e.g., logging information, file cache
- **+ Threads are more efficient than processes**
- **– Unintentional sharing can introduce subtle and hard-to-reproduce errors!**
 - The ease with which data can be shared is both the greatest strength and the greatest weakness of threads
 - Hard to know which data shared & which private
 - Hard to detect by testing
 - Probability of bad race outcome very low
 - But nonzero!
 - Future lectures

Approaches to Concurrency

■ Processes

- Hard to share resources: Easy to avoid unintended sharing
- High overhead in adding/removing clients

■ Threads

- Easy to share resources: Perhaps too easy
- Medium overhead
- Not much control over scheduling policies
- Difficult to debug
 - Event orderings not repeatable

■ I/O Multiplexing

- Tedious and low level
- Total control over scheduling
- Very low overhead
- Cannot create as fine grained a level of concurrency
- Does not make use of multi-core